CURSO CRIPTOGRAFÍA

SUSCRÍBETE

CIFRADO SIMÉTRICO

EN FLUJO

Drovohub

Algoritmo de cifrado

Algoritmo de descifrado

Existen dos tipos de cifrado simétrico

FLUJO

Va cifrando bit a bit sobre la marcha

BLOQUE

Corta el mensaje en trozos de tamaño fijo y cifra cada trozo

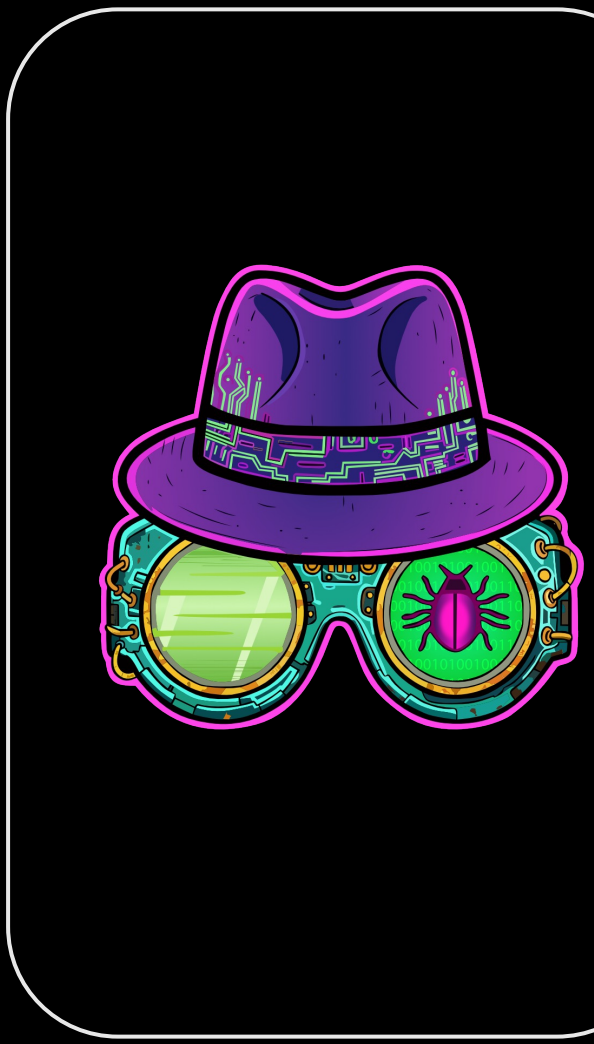
Algoritmo de cifrado

Algoritmo de descifrado

FLUJO

Va cifrando bit a bit sobre la marcha

Utilizado cuando no se conoce el tamaño de los datos previamente



Cifrado en flujo

A partir de una semilla pequeña, generamos un keystream tan largo como sea necesario

Cifrado en flujo

SEMILLA → GENERADOR → KEYSTREAM

Y después simplemente aplicamos un XOR del keystream al mensaje

MENSAJE XOR KEYSTREAM = CIFRADO

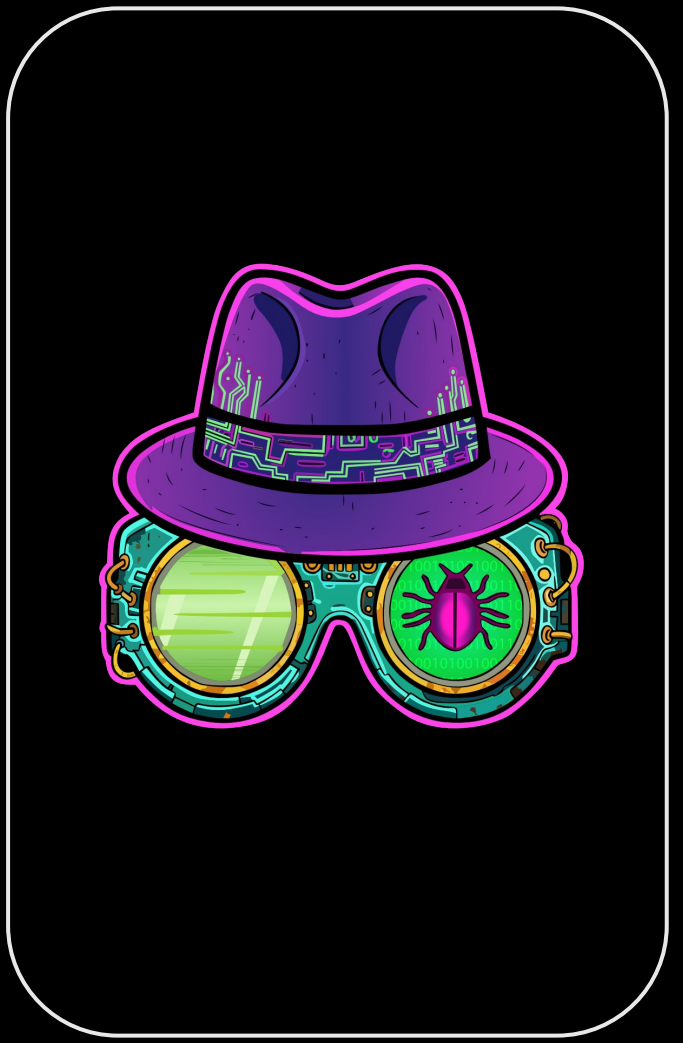
Cifrado en flujo

Pero para entenderlo bien, vamos a ver como funcionan 3 algoritmos de cifrado en flujo.

RC4

Salsa20

ChaCha20



Cifrado en flujo RC4

Cifrado RC4 (el clásico)

Diseñado en 1994. Es muy famoso históricamente, usado en WEP, WPA inicial y TLS antiguo. Hoy está obsoleto e inseguro.

Y para entenderlo veremos:

Key Scheduling Algorithm

Algoritmo que inicializa y mezcla el estado interno del cifrador usando una clave secreta.

Keystream Generation Algorithm

Algoritmo que genera la secuencia pseudoaleatoria usada para cifrar o descifrar, actualizando el estado interno.

Cifrado en flujo RC4

Key

Key Scheduling Algorithm

Keystream Generation Algorithm

MENSAJE XOR Keystream = CIFRADO

Cifrado en flujo RC4

Key Scheduling Algorithm

```
for i from 0 to 255
    S[i] := i
endfor
j := 0
for i from 0 to 255
    j := (j + S[i] + key[i mod keylength]) mod 256
    swap values of S[i] and S[j]
endfor
```

Cifrado en flujo RC4

Key Scheduling Algorithm

```
for i from 0 to 255
    S[i] := i
endfor
```

S (256 bytes)

0	1	2	3	. . .	253	254	255

Cifrado en flujo RC4

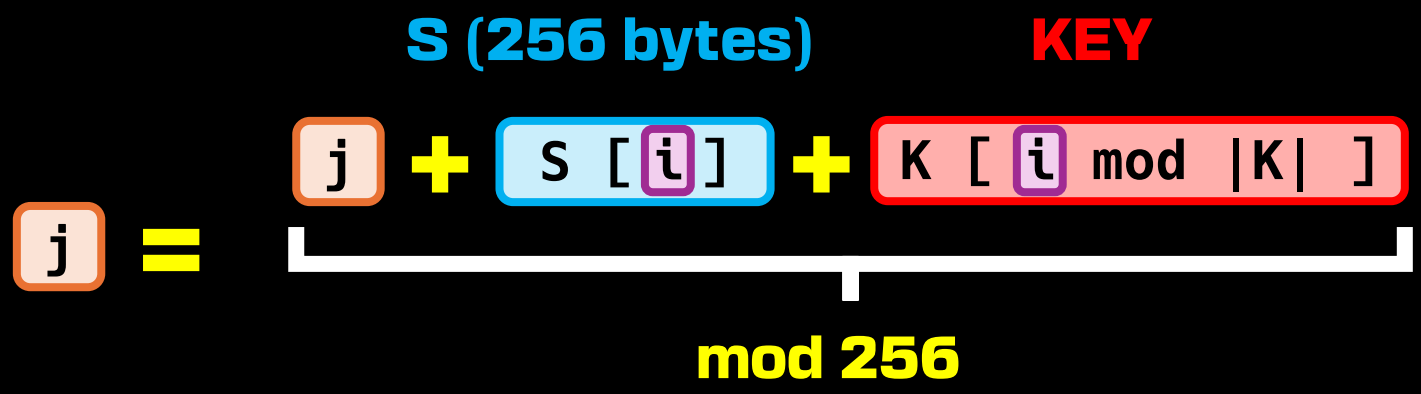
Key Scheduling Algorithm

```
for i from 0 to 255
    S[i] := i
endfor
j := 0
for i from 0 to 255
    j := (j + S[i] + key[i mod keylength]) mod 256
    swap values of S[i] and S[j]
endfor
```

Cifrado en flujo RC4

Key Scheduling Algorithm

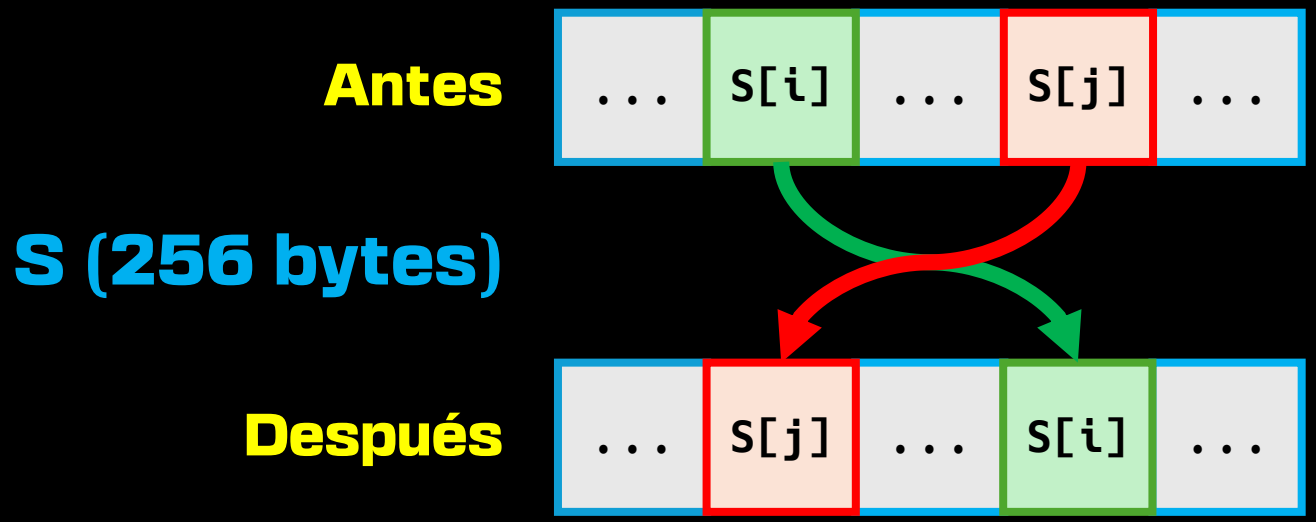
```
j := 0
for i from 0 to 255
    j := (j + S[i] + key[i mod keylength]) mod 256
    swap values of S[i] and S[j]
endfor
```

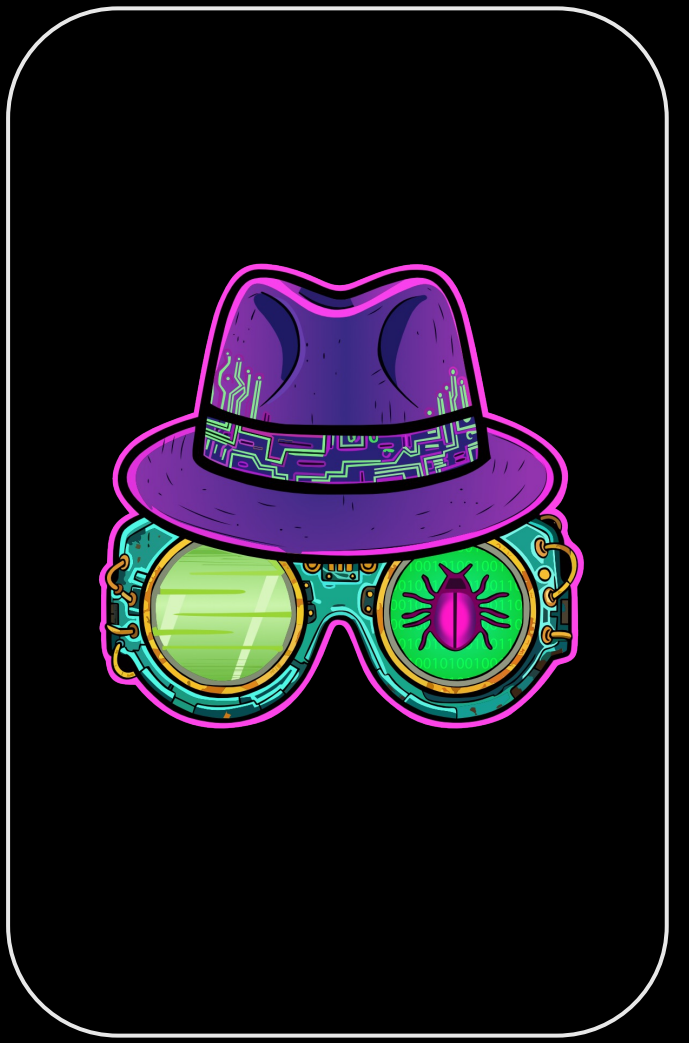


Cifrado en flujo RC4

Key Scheduling Algorithm

```
j := 0
for i from 0 to 255
    j := (j + S[i] + key[i mod keylength]) mod 256
    swap values of S[i] and S[j]
endfor
```





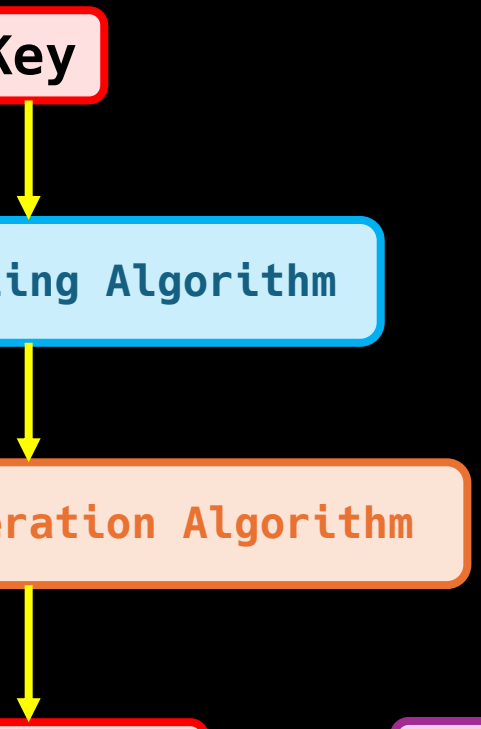
Cifrado en flujo RC4

Key Scheduling Algorithm

```
for i from 0 to 255
    S[i] := i
endfor
j := 0
for i from 0 to 255
    j := (j + S[i] + key[i mod keylength]) mod 256
    swap values of S[i] and S[j]
endfor
```

Con esto dejamos listo el estado inicial de S

Cifrado en flujo RC4



MENSAJE **XOR** Keystream = CIFRADO

Cifrado en flujo RC4

Keystream Generation Algorithm

```
i := 0
j := 0
while GeneratingOutput:
    i := (i + 1) mod 256
    j := (j + S[i]) mod 256
    swap values of S[i] and S[j]
    t := (S[i] + S[j]) mod 256
    K := S[t]
    output K
endwhile
```

Cifrado en flujo RC4

Keystream Generation Algorithm

```
i := (i + 1) mod 256
j := (j + S[i]) mod 256
```

$$i = \underbrace{i + 1}_{\text{mod } 256}$$

$$j = \underbrace{j + S[i]}_{\text{mod } 256}$$

Cifrado en flujo RC4

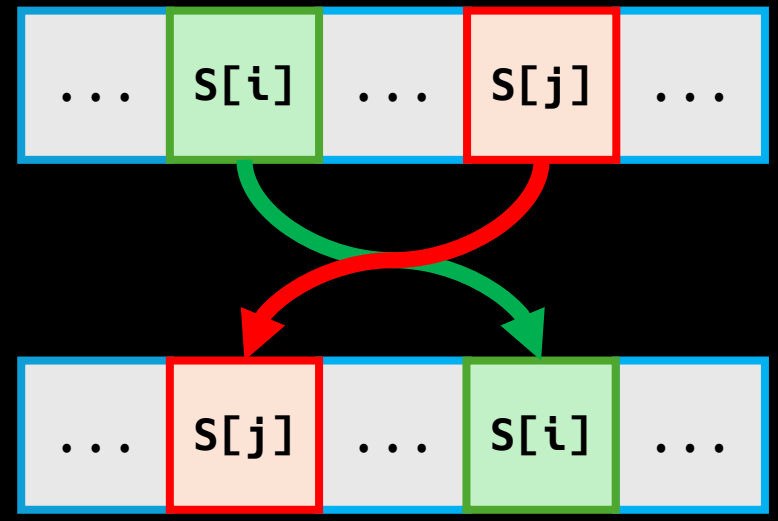
Keystream Generation Algorithm

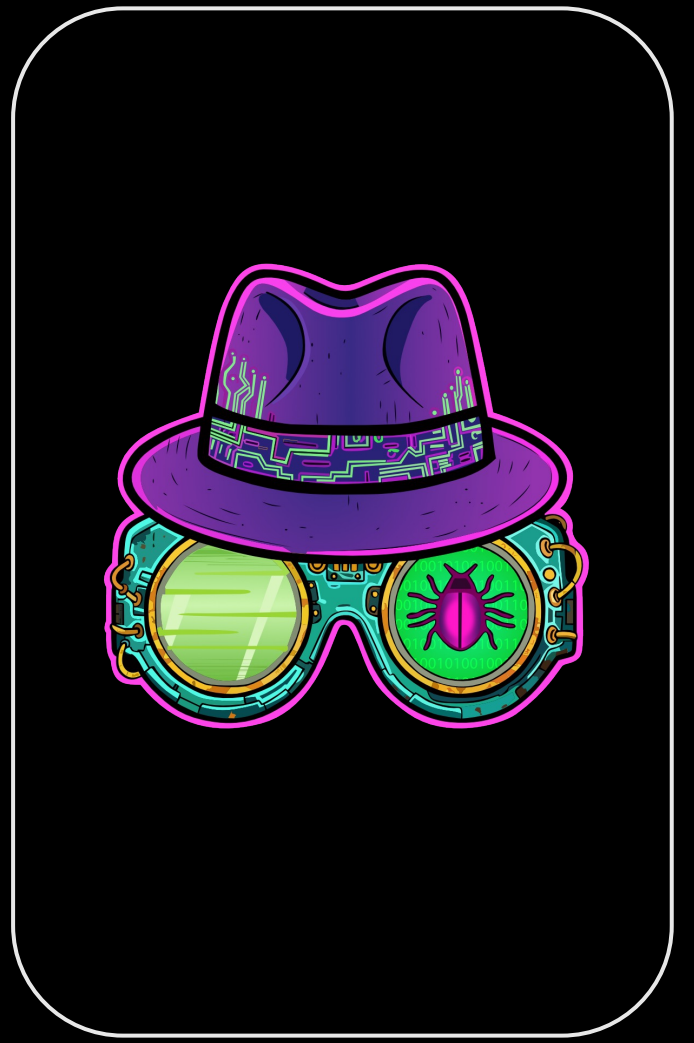
```
i := 0
j := 0
while GeneratingOutput:
    i := (i + 1) mod 256
    j := (j + S[i]) mod 256
    swap values of S[i] and S[j]
    t := (S[i] + S[j]) mod 256
    K := S[t]
    output K
endwhile
```

Cifrado en flujo RC4

Keystream Generation Algorithm

```
swap values of S[i] and S[j]
```





Cifrado en flujo RC4

Keystream Generation Algorithm

```
i := 0
j := 0
while GeneratingOutput:
    i := (i + 1) mod 256
    j := (j + S[i]) mod 256
    swap values of S[i] and S[j]
    t := (S[i] + S[j]) mod 256
    K := S[t]
    output K
endwhile
```

Cifrado en flujo RC4

Keystream Generation Algorithm

```
t := (S[i] + S[j]) mod 256
K := S[t]
output K
```

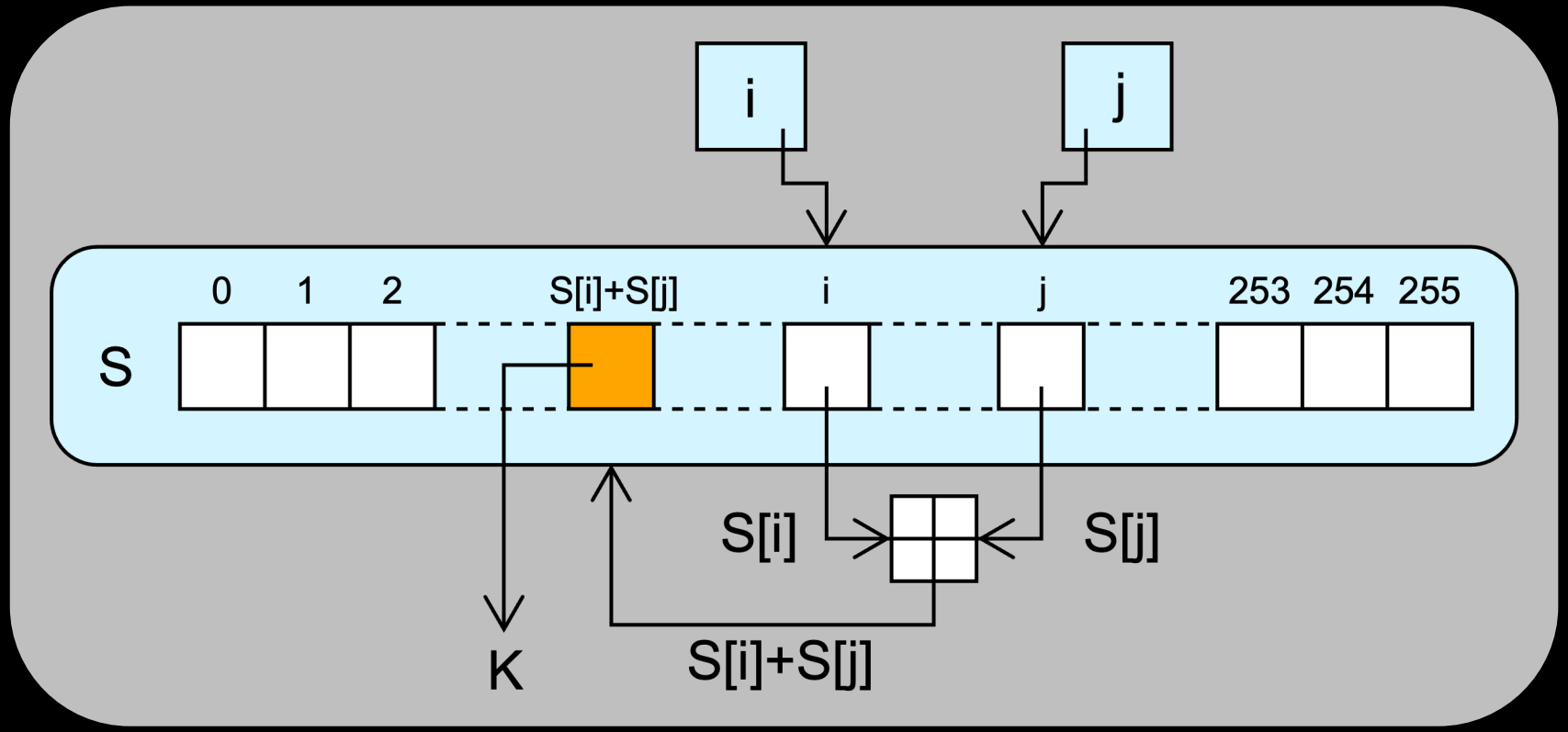
$$t = \underbrace{S[i] + S[j]}_{\text{mod } 256}$$

$$K = S[t]$$

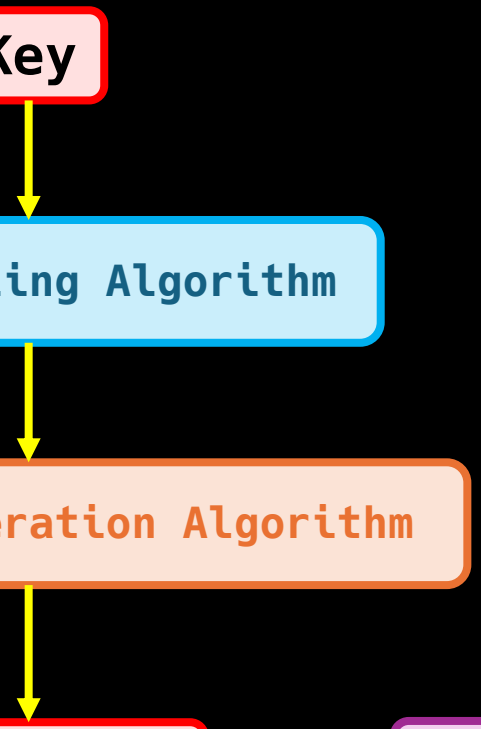
KEYSTREAM

Cifrado en flujo RC4

Keystream Generation Algorithm



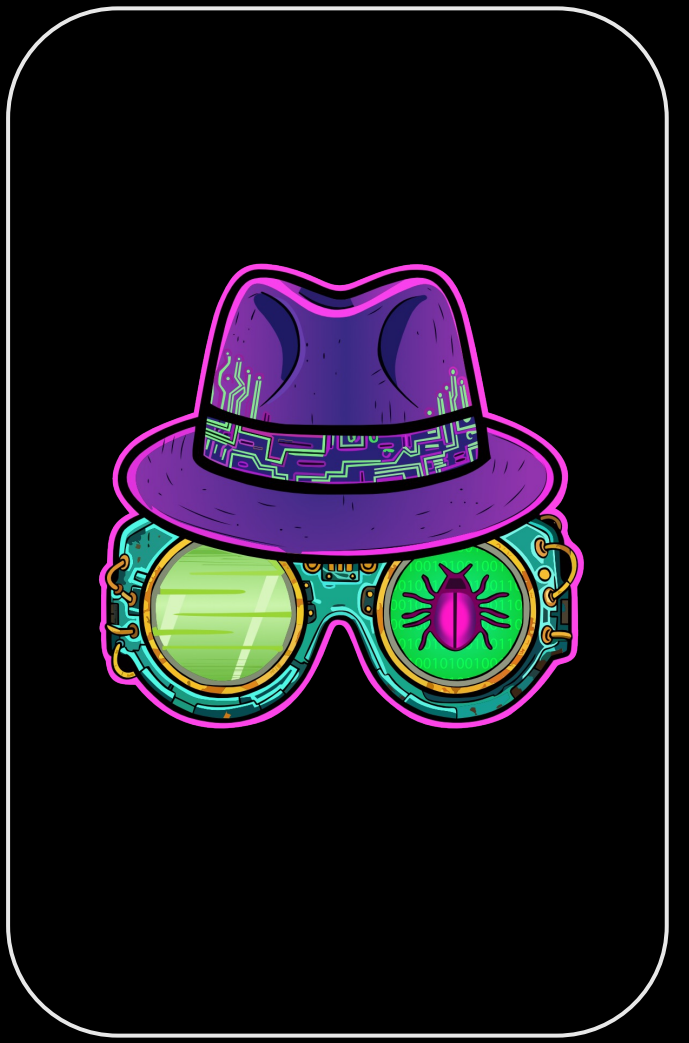
Cifrado en flujo RC4



MENSAJE **XOR** Keystream = CIFRADO

Cifrado en flujo

- RC4 ✓
- Salsa20
- ChaCha20



Cifrado en flujo Salsa20

Cifrado Salsa20

Diseñado en 2005. Es un cifrador de flujo moderno y eficiente, conocido por su buen rendimiento en software.

Y para entenderlo veremos:

Initial State Assembly

Algoritmo que construye el estado inicial usando constantes, clave secreta, nonce y contador de bloque.

Keystream Block Generation

Algoritmo que aplica la función de rondas de Salsa20 al estado inicial para generar bloques de keystream usados para cifrar o descifrar.

Cifrado en flujo Salsa20

Key

Nonce

Contador

Initial State Assembly

Construir estado inicial S

Keystream Block Generation

Copiar estado inicial

20 rondas Salsa20

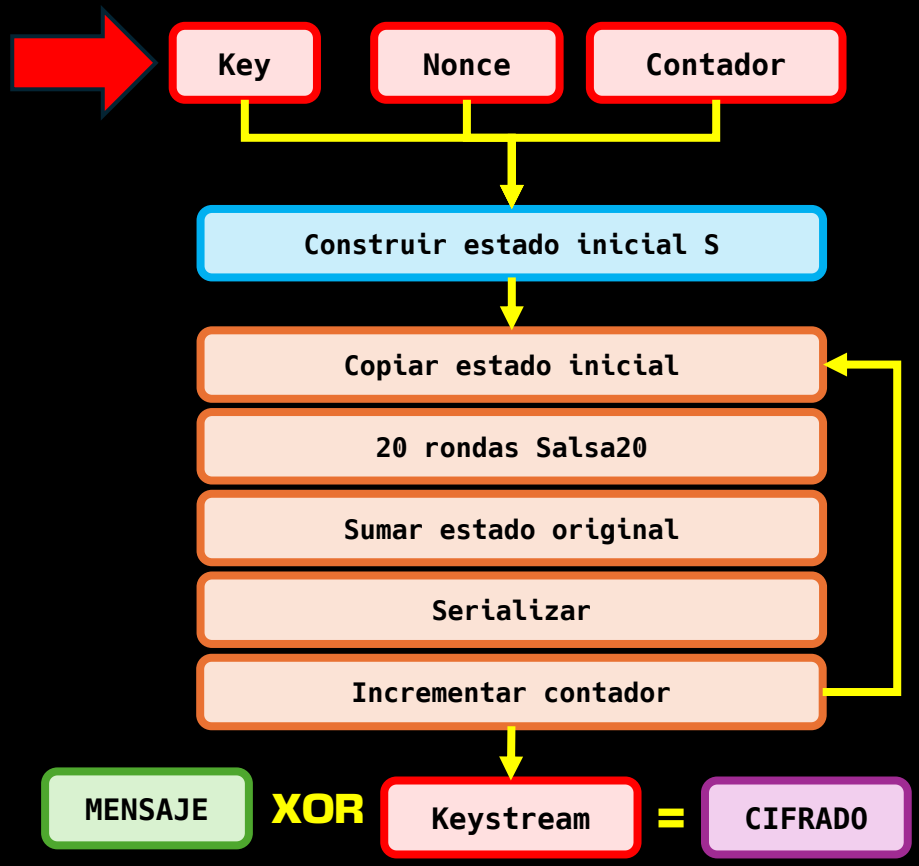
Sumar estado original

Serializar

Incrementar contador

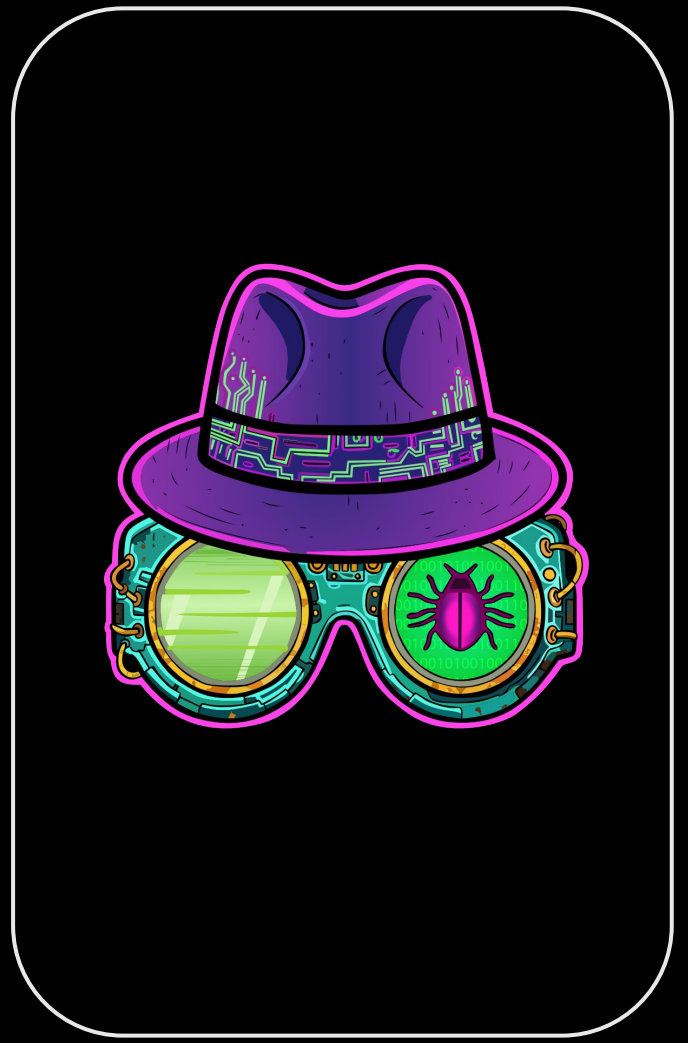
MENSAJE XOR Keystream = CIFRADO

Cifrado en flujo Salsa20




```
Salsa20_Cipher(key[32 bytes], nonce[8 bytes]):
    counter := 0

    while GeneratingOutput:
        state := BuildInitialState(key, nonce, counter)
```

Cifrado en flujo Salsa20



Key

Nonce

Contador

Initial State Assembly

Construir estado inicial S

Keystream Block Generation

Copiar estado inicial

20 rondas Salsa20

Sumar estado original

Serializar

Incrementar contador

MENSAJE XOR Keystream = CIFRADO

Cifrado en flujo Salsa20

Initial State Assembly

Construimos la la matriz S

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Cifrado en flujo Salsa20

Initial State Assembly

```
BuildInitialState(key[32 bytes], nonce[8 bytes], counter):

    constants := "expand 32-byte k"

    input[0]  := littleendian(constants[0..3])
    input[1]  := littleendian(key[0..3])
    input[2]  := littleendian(key[4..7])
    input[3]  := littleendian(key[8..11])
    input[4]  := littleendian(key[12..15])

    input[5]  := littleendian(constants[4..7])
    input[6]  := littleendian(nonce[0..3])
    input[7]  := littleendian(nonce[4..7])
    input[8]  := lower 32 bits of counter
    input[9]  := upper 32 bits of counter

    input[10] := littleendian(constants[8..11])
    input[11] := littleendian(key[16..19])
    input[12] := littleendian(key[20..23])
    input[13] := littleendian(key[24..27])
    input[14] := littleendian(key[28..31])

    input[15] := littleendian(constants[12..15])

    return input
```

Matriz S

expa			
	nd 3		
		2-by	
			te k

Cifrado en flujo Salsa20

Initial State Assembly

```
BuildInitialState(key[32 bytes], nonce[8 bytes], counter):

    constants := "expand 32-byte k"

    input[0]  := littleendian(constants[0..3])
    input[1]  := littleendian(key[0..3])
    input[2]  := littleendian(key[4..7])
    input[3]  := littleendian(key[8..11])
    input[4]  := littleendian(key[12..15])

    input[5]  := littleendian(constants[4..7])
    input[6]  := littleendian(nonce[0..3])
    input[7]  := littleendian(nonce[4..7])
    input[8]  := lower 32 bits of counter
    input[9]  := upper 32 bits of counter

    input[10] := littleendian(constants[8..11])
    input[11] := littleendian(key[16..19])
    input[12] := littleendian(key[20..23])
    input[13] := littleendian(key[24..27])
    input[14] := littleendian(key[28..31])

    input[15] := littleendian(constants[12..15])

    return input
```

Matriz S

expa	k_0	k_1	k_2
k_3	nd 3		
		2-by	k_4
k_5	k_6	k_7	te k

Cifrado en flujo Salsa20

Initial State Assembly

```
BuildInitialState(key[32 bytes], nonce[8 bytes], counter):

    constants := "expand 32-byte k"

    input[0]  := littleendian(constants[0..3])
    input[1]  := littleendian(key[0..3])
    input[2]  := littleendian(key[4..7])
    input[3]  := littleendian(key[8..11])
    input[4]  := littleendian(key[12..15])

    input[5]  := littleendian(constants[4..7])
    input[6]  := littleendian(nonce[0..3])
    input[7]  := littleendian(nonce[4..7])
    input[8]  := lower 32 bits of counter
    input[9]  := upper 32 bits of counter

    input[10] := littleendian(constants[8..11])
    input[11] := littleendian(key[16..19])
    input[12] := littleendian(key[20..23])
    input[13] := littleendian(key[24..27])
    input[14] := littleendian(key[28..31])

    input[15] := littleendian(constants[12..15])

    return input
```

Matriz S

expa	k_0	k_1	k_2
k_3	nd 3	nonce_0	nonce_1
		2-by	k_4
k_5	k_6	k_7	te k

Cifrado en flujo Salsa20

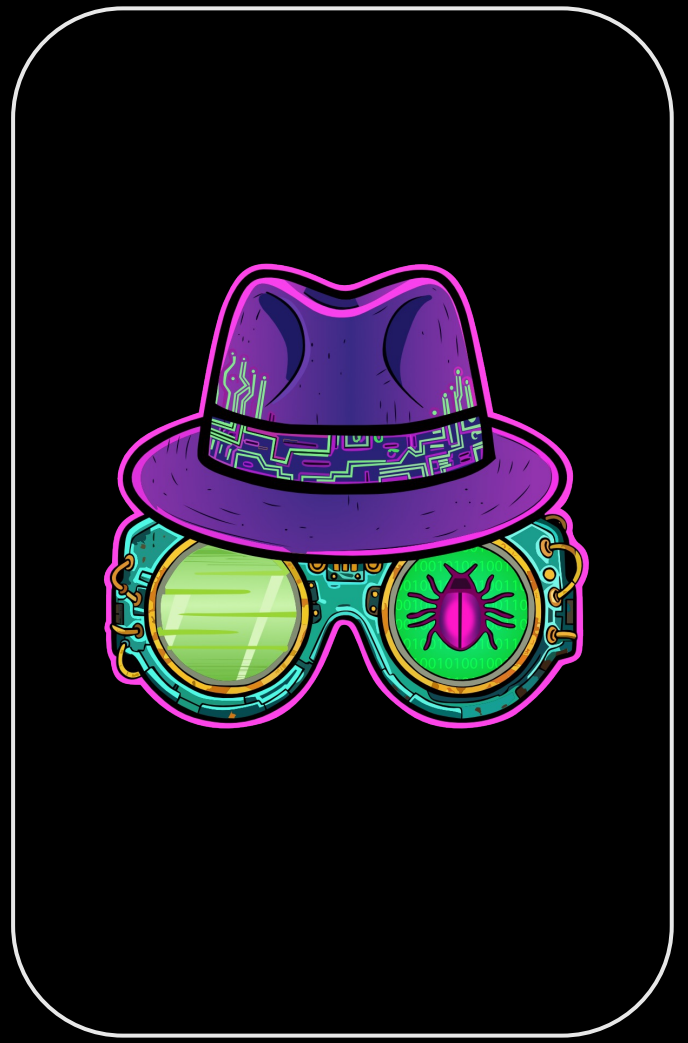
Initial State Assembly

```
BuildInitialState(key[32 bytes], nonce[8 bytes], counter):

    constants := "expand 32-byte k"

    input[0]  := littleendian(constants[0..3])
    input[1]  := littleendian(key[0..3])
    input[2]  := littleendian(key[4..7])
    input[3]  := littleendian(key[8..11])
    input[4]  := littleendian(key[12..15])

    input[5]  := littleendian(constants[4..7])
    input[6]  := littleendian(nonce[0..3])
    input[7]  := littleendian(nonce[4..7])
    input[8]  := lower 32 bits of counter
    input[9]  := upper 32 bits of counter

    input[10] := littleendian(constants[8..11])
    input[11] := littleendian(key[16..19])
    input[12] := littleendian(key[20..23])
    input[13] := littleendian(key[24..27])
    input[14] := littleendian(key[28..31])

    input[15] := littleendian(constants[12..15])

    return input
```

Matriz S

expa	k_0	k_1	k_2
k_3	nd 3	nonce_0	nonce_1
counter_0	counter_1	2-by	k_4
k_5	k_6	k_7	te k

Cifrado en flujo Salsa20



Key

Nonce

Contador

Initial State Assembly

Construir estado inicial S

Copiar estado inicial

20 rondas Salsa20

Sumar estado original

Serializar

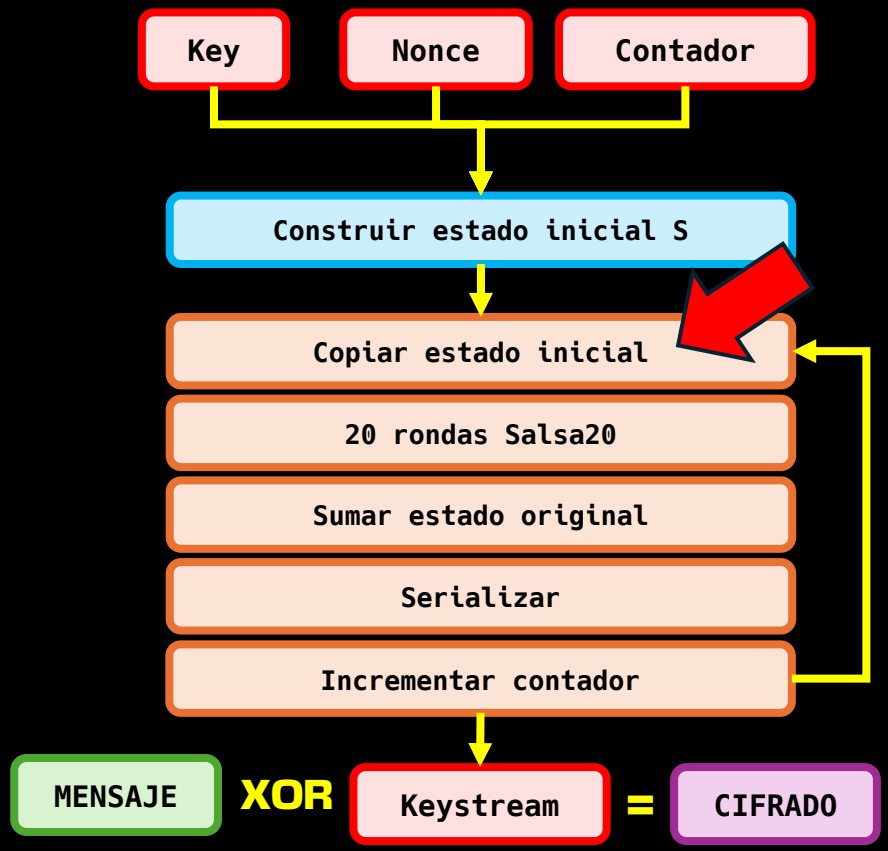
Incrementar contador

Keystream Block Generation

MENSAJE XOR Keystream = CIFRADO

Cifrado en flujo Salsa20

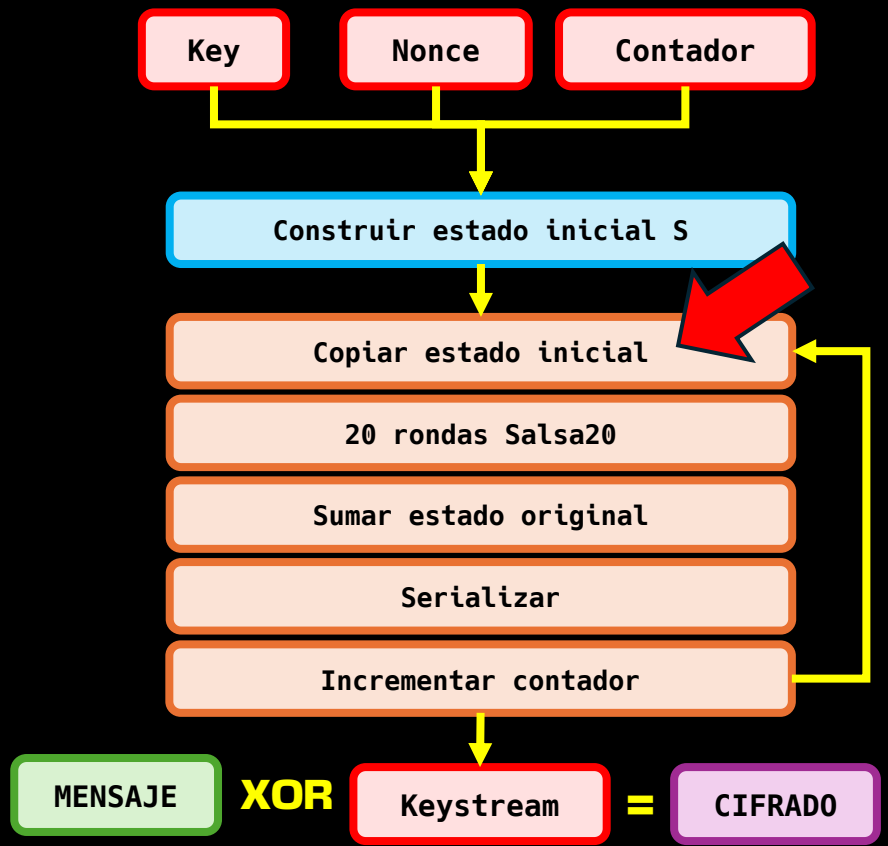
Keystream Generation Algorithm




```
Salsa20_Cipher(key[32 bytes], nonce[8 bytes]):
    counter := 0

    while GeneratingOutput:
        state := BuildInitialState(key, nonce, counter)

        for i from 0 to 15
            x[i] := state[i]
        endfor
```

Cifrado en flujo Salsa20

Keystream Generation Algorithm



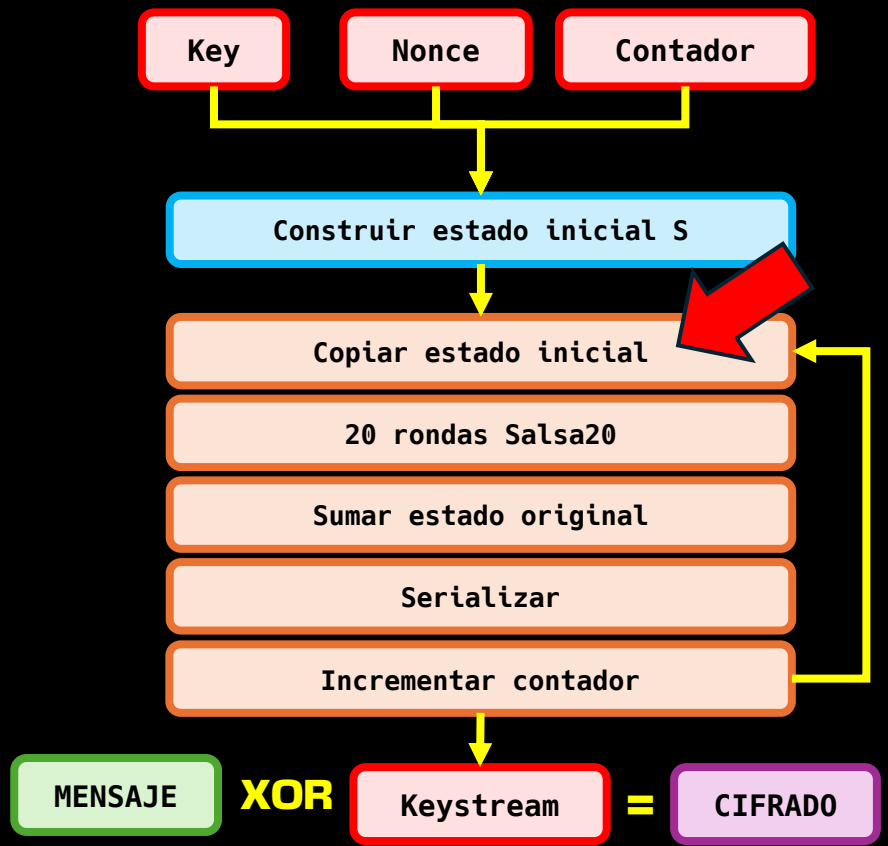

```
Salsa20_Cipher(key[32 bytes], nonce[8 bytes]):
    counter := 0

    while GeneratingOutput:
        state := BuildInitialState(key, nonce, counter)

        for i from 0 to 15
            x[i] := state[i]
        endfor
```

Copiamos la matriz S

"expa"	k_0	k_1	k_2
k_3	"nd 3"	nonce_0	nonce_1
counter_0	counter_1	"2-by"	k_4
k_5	k_6	k_7	"te k"

Cifrado en flujo Salsa20

Keystream Generation Algorithm



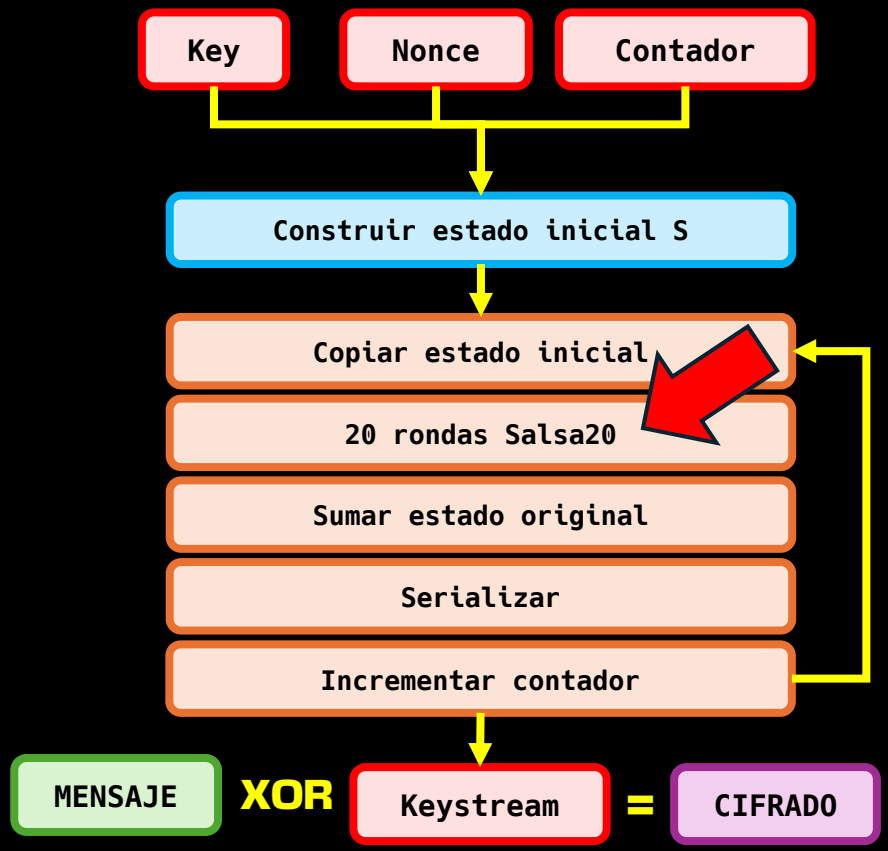

```
Salsa20_Cipher(key[32 bytes], nonce[8 bytes]):
    counter := 0

    while GeneratingOutput:
        state := BuildInitialState(key, nonce, counter)

        for i from 0 to 15
            x[i] := state[i]
        endfor
```

En la matriz de trabajo X

“expa”	k_0	k_1	k_2
k_3	“nd 3”	nonce_0	nonce_1
counter_0	counter_1	“2-by”	k_4
k_5	k_6	k_7	“te k”

Cifrado en flujo Salsa20

Keystream Generation Algorithm




```
Salsa20_Cipher(key[32 bytes], nonce[8 bytes]):
    counter := 0

    while GeneratingOutput:
        state := BuildInitialState(key, nonce, counter)

        for i from 0 to 15
            x[i] := state[i]
        endfor

        for round from 0 to 9
            QuarterRound(x[0],  x[4],  x[8],  x[12])
            QuarterRound(x[5],  x[9],  x[13], x[1])
            QuarterRound(x[10], x[14], x[2],  x[6])
            QuarterRound(x[15], x[3],  x[7],  x[11])

            QuarterRound(x[0],  x[1],  x[2],  x[3])
            QuarterRound(x[5],  x[6],  x[7],  x[4])
            QuarterRound(x[10], x[11], x[8],  x[9])
            QuarterRound(x[15], x[12], x[13], x[14])
        endfor
```

Cifrado en flujo Salsa20

Keystream Generation Algorithm

20 rondas Salsa20

```
for round from 0 to 9
    QuarterRound(x[0],  x[4],  x[8],  x[12])
    QuarterRound(x[5],  x[9],  x[13], x[1])
    QuarterRound(x[10], x[14], x[2],  x[6])
    QuarterRound(x[15], x[3],  x[7],  x[11])

    QuarterRound(x[0],  x[1],  x[2],  x[3])
    QuarterRound(x[5],  x[6],  x[7],  x[4])
    QuarterRound(x[10], x[11], x[8],  x[9])
    QuarterRound(x[15], x[12], x[13], x[14])
endfor
```

Cifrado en flujo Salsa20

Keystream Generation Algorithm

20 rondas Salsa20

```
QuarterRound(a, b, c, d):
    b := b XOR ROTL(a + d mod 2^32, 7)
    c := c XOR ROTL(b + a mod 2^32, 9)
    d := d XOR ROTL(c + b mod 2^32, 13)
    a := a XOR ROTL(d + c mod 2^32, 18)
```

Recibe 4 palabras de 32 bits y se encarga de mezclarlas mediante tres operaciones:

- Suma módulo 2^32
- Rotación izquierda
- XOR

Cifrado en flujo Salsa20

Keystream Generation Algorithm

20 rondas Salsa20

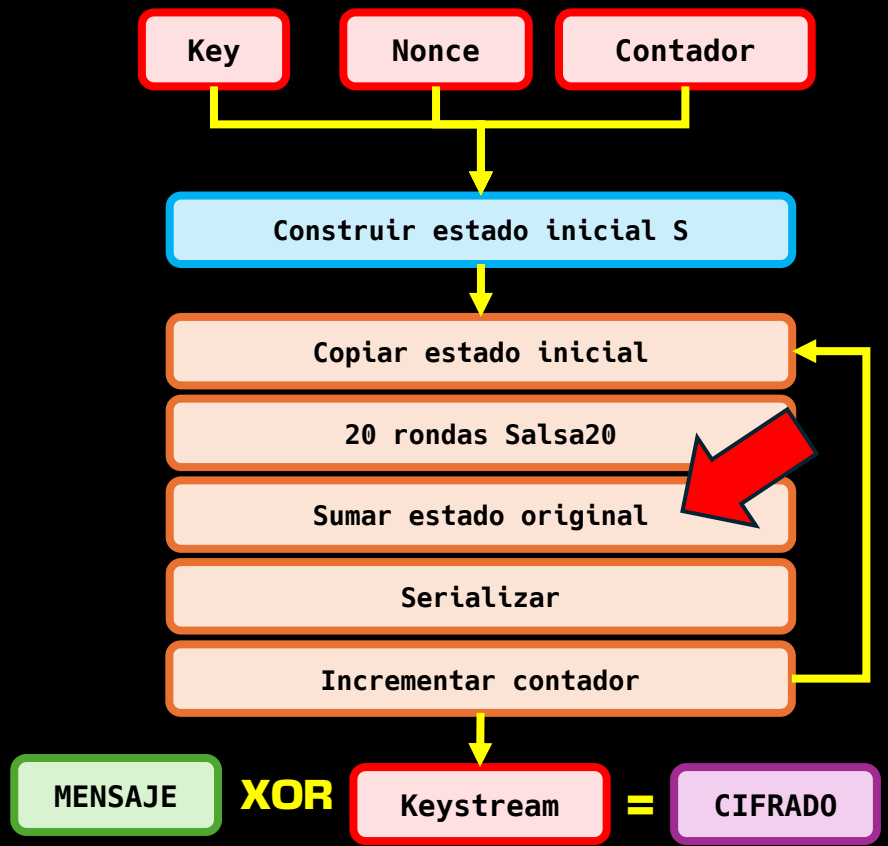
```
for round from 0 to 9
    QuarterRound(x[0],  x[4],  x[8],  x[12])
    QuarterRound(x[5],  x[9],  x[13], x[1])
    QuarterRound(x[10], x[14], x[2],  x[6])
    QuarterRound(x[15], x[3],  x[7],  x[11])

    QuarterRound(x[0],  x[1],  x[2],  x[3])
    QuarterRound(x[5],  x[6],  x[7],  x[4])
    QuarterRound(x[10], x[11], x[8],  x[9])
    QuarterRound(x[15], x[12], x[13], x[14])
endfor
```

Columnas

Filas

Cifrado en flujo Salsa20

Keystream Generation Algorithm




```
Salsa20_Cipher(key[32 bytes], nonce[8 bytes]):
    counter := 0

    while GeneratingOutput:
        state := BuildInitialState(key, nonce, counter)

        for i from 0 to 15
            x[i] := state[i]
        endfor

        for round from 0 to 9
            QuarterRound(x[0],  x[4],  x[8],  x[12])
            QuarterRound(x[5],  x[9],  x[13], x[1])
            QuarterRound(x[10], x[14], x[2],  x[6])
            QuarterRound(x[15], x[3],  x[7],  x[11])

            QuarterRound(x[0],  x[1],  x[2],  x[3])
            QuarterRound(x[5],  x[6],  x[7],  x[4])
            QuarterRound(x[10], x[11], x[8],  x[9])
            QuarterRound(x[15], x[12], x[13], x[14])
        endfor

        for i from 0 to 15
            block[i] := x[i] + state[i] mod 2^32
        endfor
```

Cifrado en flujo Salsa20

Keystream Generation Algorithm

Sumar estado original

```
for i from 0 to 15
    block[i] := x[i] + state[i] mod 2^32
endfor
```

$$B[i] = \underbrace{X[i] + S[i]}_{\text{mod } 256}$$

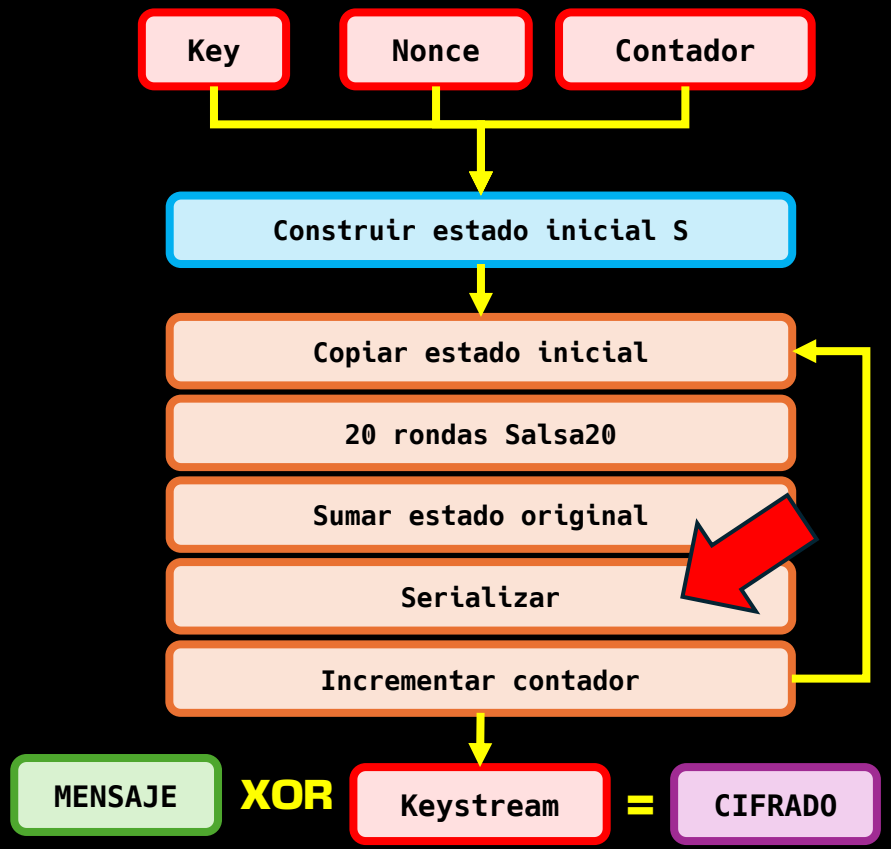
Matriz B

Matriz X

Matriz S

Cifrado en flujo Salsa20

Keystream Generation Algorithm




```
Salsa20_Cipher(key[32 bytes], nonce[8 bytes]):
    counter := 0

    while GeneratingOutput:
        state := BuildInitialState(key, nonce, counter)

        for i from 0 to 15
            x[i] := state[i]
        endfor

        for round from 0 to 9
            QuarterRound(x[0],  x[4],  x[8],  x[12])
            QuarterRound(x[5],  x[9],  x[13], x[1])
            QuarterRound(x[10], x[14], x[2],  x[6])
            QuarterRound(x[15], x[3],  x[7],  x[11])

            QuarterRound(x[0],  x[1],  x[2],  x[3])
            QuarterRound(x[5],  x[6],  x[7],  x[4])
            QuarterRound(x[10], x[11], x[8],  x[9])
            QuarterRound(x[15], x[12], x[13], x[14])
        endfor

        for i from 0 to 15
            block[i] := x[i] + state[i] mod 2^32
        endfor

        for i from 0 to 15
            output block[i] as 4 little-endian bytes
        endfor
```

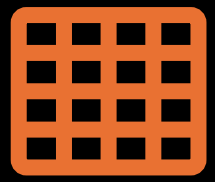
Cifrado en flujo Salsa20

Keystream Generation Algorithm

Serializar

```
for i from 0 to 15
    output block[i] as 4 little-endian bytes
endfor
```

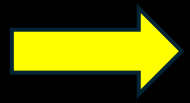
Matriz B



= 16 x

8 bytes

32 bits

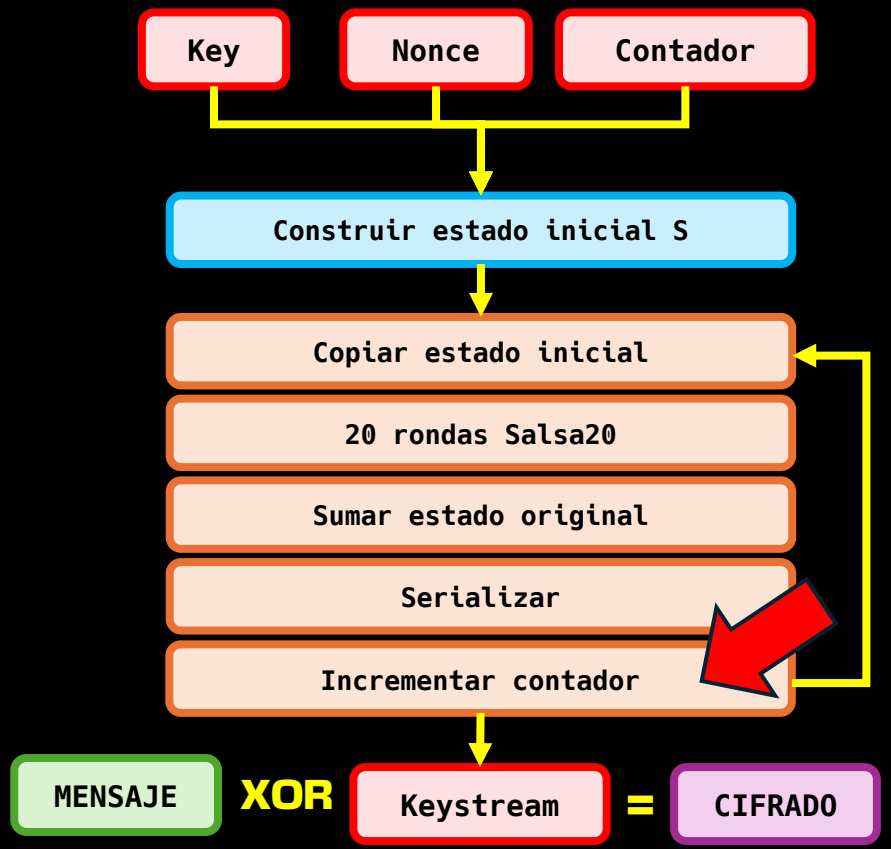


Bytes de uno en uno

keystream

Cifrado en flujo Salsa20

Keystream Generation Algorithm




```
Salsa20_Cipher(key[32 bytes], nonce[8 bytes]):
    counter := 0

    while GeneratingOutput:
        state := BuildInitialState(key, nonce, counter)

        for i from 0 to 15
            x[i] := state[i]
        endfor

        for round from 0 to 9
            QuarterRound(x[0],  x[4],  x[8],  x[12])
            QuarterRound(x[5],  x[9],  x[13], x[1])
            QuarterRound(x[10], x[14], x[2],  x[6])
            QuarterRound(x[15], x[3],  x[7],  x[11])

            QuarterRound(x[0],  x[1],  x[2],  x[3])
            QuarterRound(x[5],  x[6],  x[7],  x[4])
            QuarterRound(x[10], x[11], x[8],  x[9])
            QuarterRound(x[15], x[12], x[13], x[14])
        endfor

        for i from 0 to 15
            block[i] := x[i] + state[i] mod 2^32
        endfor

        for i from 0 to 15
            output block[i] as 4 little-endian bytes
        endfor

        counter := counter + 1
    endwhile
```

Cifrado en flujo Salsa20

Keystream Generation Algorithm

Incrementar contador

```
    counter := counter + 1
endwhile
```

counter_1 counter_0 += 1

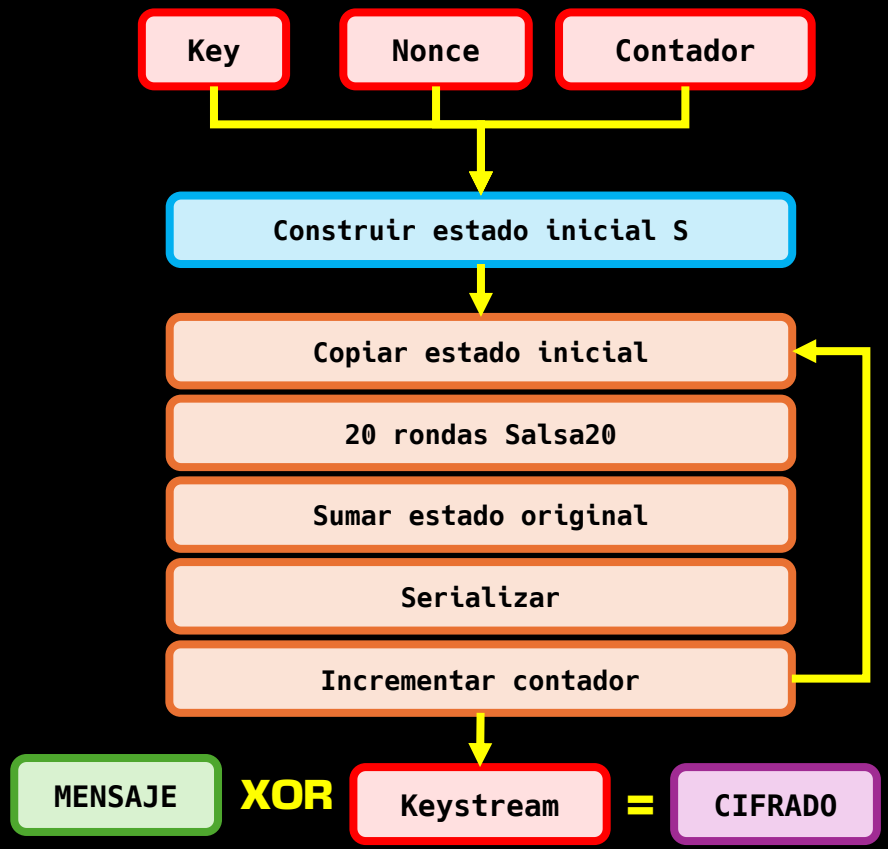
En el siguiente ciclo...

Matriz S

“expa”	k_0	k_1	k_2
k_3	“nd 3”	nonce_0	nonce_1
counter_0	counter_1	“2-by”	k_4
k_5	k_6	k_7	“te k”

Cifrado en flujo Salsa20

Keystream Generation Algorithm




```
Salsa20_Cipher(key[32 bytes], nonce[8 bytes]):
    counter := 0

    while GeneratingOutput:
        state := BuildInitialState(key, nonce, counter)

        for i from 0 to 15
            x[i] := state[i]
        endfor

        for round from 0 to 9
            QuarterRound(x[0],  x[4],  x[8],  x[12])
            QuarterRound(x[5],  x[9],  x[13], x[1])
            QuarterRound(x[10], x[14], x[2],  x[6])
            QuarterRound(x[15], x[3],  x[7],  x[11])

            QuarterRound(x[0],  x[1],  x[2],  x[3])
            QuarterRound(x[5],  x[6],  x[7],  x[4])
            QuarterRound(x[10], x[11], x[8],  x[9])
            QuarterRound(x[15], x[12], x[13], x[14])
        endfor

        for i from 0 to 15
            block[i] := x[i] + state[i] mod 2^32
        endfor

        for i from 0 to 15
            output block[i] as 4 little-endian bytes
        endfor

        counter := counter + 1
    endwhile
```

Cifrado en flujo Salsa20

Key

Nonce

Contador

Initial State Assembly

Construir estado inicial S

Keystream Block Generation

Copiar estado inicial

20 rondas Salsa20

Sumar estado original

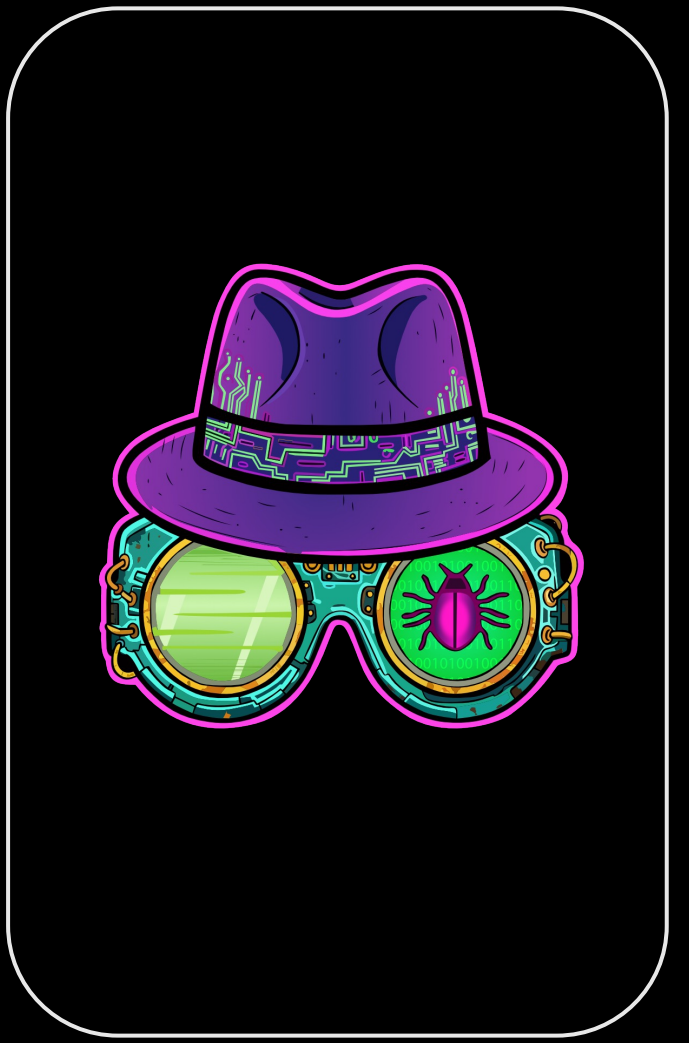
Serializar

Incrementar contador

MENSAJE XOR Keystream = CIFRADO

Cifrado en flujo

- RC4 ✓
- Salsa20 ✓
- ChaCha20



Cifrado en flujo ChaCha20

Cifrado ChaCha20 (el moderno)

Publicado en 2008, es el sucesor de Salsa20 y muy usado actualmente. Rápido, seguro y popular en TLS, VPNs y sistemas modernos.

Es muy similar a Salsa20, a alto nivel, solo cambian tres elementos:

- `Estado inicial`
- `Patrón rondas`
- `Quarterround`

Cifrado en flujo ChaCha20

Cambio 1: Estado inicial

ANTES

Matriz S

“expa”	k_0	k_1	k_2
k_3	“nd 3”	nonce_0	nonce_1
counter_0	counter_1	“2-by”	k_4
k_5	k_6	k_7	“te k”

Cifrado en flujo ChaCha20

Cambio 1: Estado inicial

DESPUÉS

Matriz S

“expa”	“nd 3”	“2-by”	“te k”
k_1	k_2	k_3	k_4
k_5	k_6	k_7	k_8
counter_0	counter_1	nonce_0	nonce_1

Cifrado en flujo ChaCha20

Cambio 2: Patrón rondas

ANTES

```
for round from 0 to 9
    QuarterRound(x[0],  x[4],  x[8],  x[12])
    QuarterRound(x[5],  x[9],  x[13], x[1])
    QuarterRound(x[10], x[14], x[2],  x[6])
    QuarterRound(x[15], x[3],  x[7],  x[11])

    QuarterRound(x[0],  x[1],  x[2],  x[3])
    QuarterRound(x[5],  x[6],  x[7],  x[4])
    QuarterRound(x[10], x[11], x[8],  x[9])
    QuarterRound(x[15], x[12], x[13], x[14])
endfor
```

Columnas

Filas

Cifrado en flujo ChaCha20

Cambio 2: Patrón rondas

DESPUÉS

```
for round from 0 to 9
    QuarterRound(x[0],  x[4],  x[8],  x[12])
    QuarterRound(x[5],  x[9],  x[13], x[1])
    QuarterRound(x[10], x[14], x[2],  x[6])
    QuarterRound(x[15], x[3],  x[7],  x[11])

    QuarterRound(x[0],  x[1],  x[2],  x[3])
    QuarterRound(x[5],  x[6],  x[7],  x[4])
    QuarterRound(x[10], x[11], x[8],  x[9])
    QuarterRound(x[15], x[12], x[13], x[14])
endfor
```

Columnas

Diagonales

Cifrado en flujo ChaCha20

Cambio 3: Quarterround

ANTES

```
QuarterRound(a, b, c, d):
    b := b XOR ROTL(a + d mod 2^32, 7)
    c := c XOR ROTL(b + a mod 2^32, 9)
    d := d XOR ROTL(c + b mod 2^32, 13)
    a := a XOR ROTL(d + c mod 2^32, 18)
```

DESPUÉS

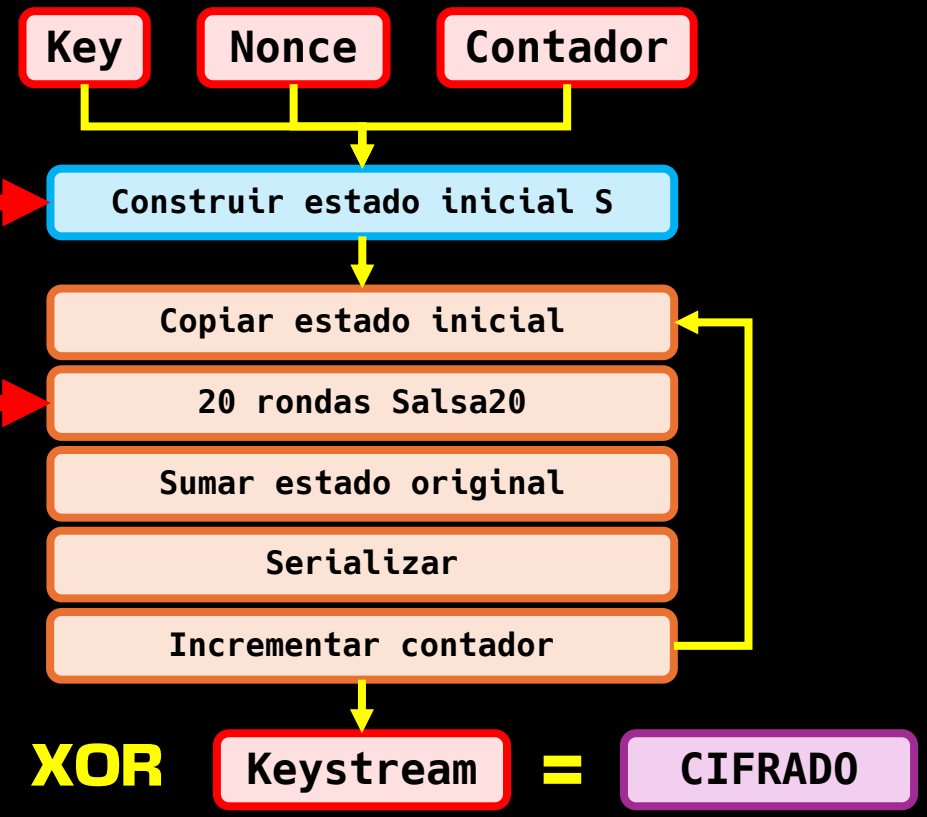
```
QuarterRound(a, b, c, d):
    a := a + b mod 2^32
    d := d XOR a
    d := ROTL(d, 16)

    c := c + d mod 2^32
    b := b XOR c
    b := ROTL(b, 12)

    a := a + b mod 2^32
    d := d XOR a
    d := ROTL(d, 8)

    c := c + d mod 2^32
    b := b XOR c
    b := ROTL(b, 7)
```

Cifrado en flujo ChaCha20

Cambio 1:
Estado inicial

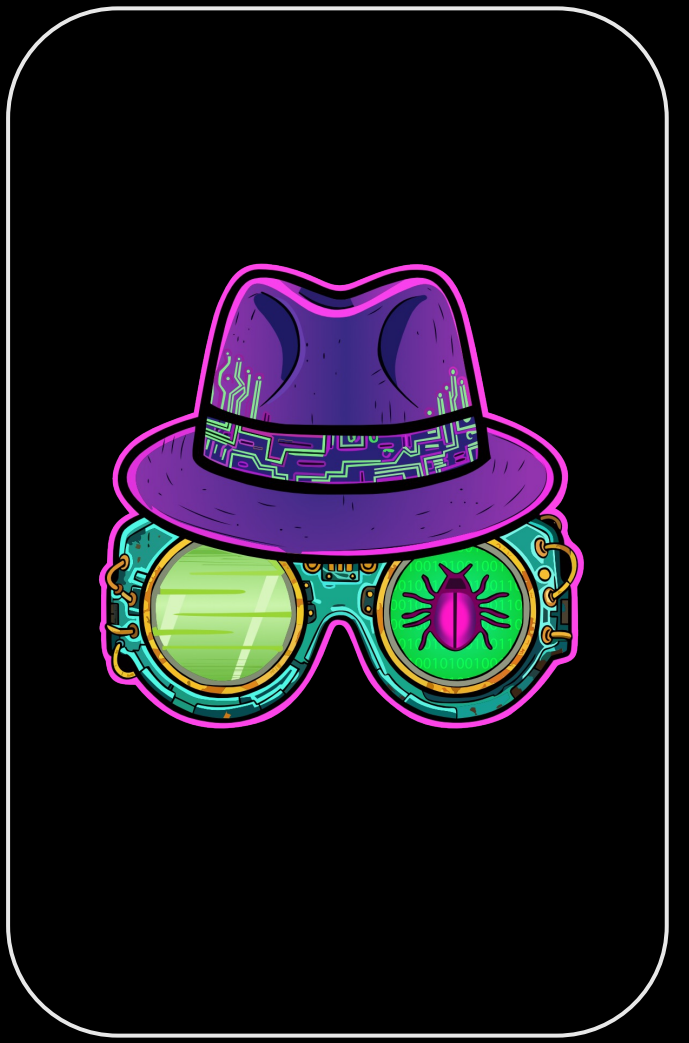
Cambio 2:
Patrón rondas

Cambio 3:
Quarterround



Cifrado en flujo

- RC4 ✓
- Salsa20 ✓
- ChaCha20 ✓



DrovoLeaks Public

Unpin | Watch 1 | Fork 2 | Star 28

main | 1 Branch | 0 Tags | Go to file T | Add file | <>

drovoh4k Nueva clase: Metodología análisis de binarios 6a7d86c · 2 weeks ago 42 Commits

cursos	Nueva clase: Metodología análisis de binarios	2 weeks ago
masterclass	Refactor: meta.json removed + Links corrected + Images c...	2 months ago
tutoriales	Refactor: meta.json removed + Links corrected + Images c...	2 months ago
varios/ida_vs_ghidra	Nuevo video: Ida VS Ghidra	4 months ago
vulnerabilidades	Nueva vulnerabilidad: CVE-2026-3909 y CVE-2026-3910 ...	last month
writeups	Nuevo CTF: Hack4uCTF (crypto) - Locked Away	last month
README.md	Nueva masterclass: Introducción a MIPS	3 months ago

README

DrovoLeaks

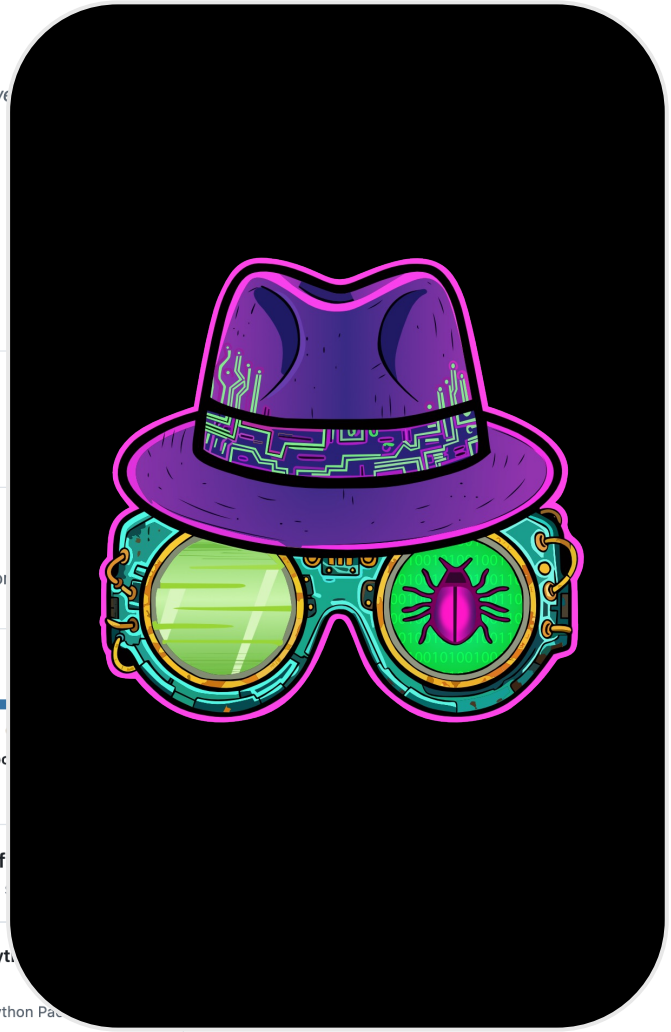
Channel @drovoh4k

Este repositorio es el **material de apoyo** de mis vídeos de YouTube.

La idea es tener todo el conocimiento del canal en un lugar centralizado y ordenado, no solo los propios videos, sino también todos los recursos extra.

🧭 Navegación rápida

- 📝 Writeups
  - Resoluciones de retos (crackmes, plataformas y CTFs), con scripts y recursos.
- 📚 Cursos
  - Contenido estructurado en módulos y clases: enfoque progresivo, con enlaces a playlist y recursos.
- 🎓 Masterclass



About

No description, we provided.

Readme
Activity
28 stars
1 watching
2 forks

Releases

No releases published
Create a new release

Contributors 1

drovoh4k Llor

Languages

Python 76.4%
C 6.2% Doc
Shell 3.5%

Suggested workf
Based on your tech

Publish Pyt Package
Publish a Python Pa on release.

Python application Configure
Create and test a Python application.

tutoriales	Refactor: meta.json removed + Links corrected + Images c...	2 months ago
varios/ida_vs_ghidra	Nuevo video: Ida VS Ghidra	4 months ago
vulnerabilidades	Nueva vulnerabilidad: CVE-2026-3909 y CVE-2026-3910 ...	last month
writeups	Nuevo CTF: Hack4uCTF (crypto) - Locked Away	last month
README.md	Nueva masterclass: Introducción a MIPS	3 months ago

README

DrovoLeaks

Channel @drovoh4k

Este repositorio es el **material de apoyo** de mis vídeos de YouTube.

La idea es tener todo el conocimiento del canal en un lugar centralizado y ordenado, no solo los propios videos, sino también todos los recursos extra.

🧭 Navegación rápida

- 📝 Writeups
  - Resoluciones de retos (crackmes, plataformas y CTFs), con scripts y recursos.
- 📚 Cursos
  - Contenido estructurado en módulos y clases: enfoque progresivo, con enlaces a playlist y recursos.
- 🎓 Masterclass
  - Clases específicas para quienes ya dominan los fundamentos: se asume base previa y se entra directamente en una temática concreta con enfoque técnico y práctico.
- ⛑️ Tutoriales
  - Guías/tips independientes sobre conceptos o tooling.
- 🐞 Vulnerabilidades
  - CVEs concretos: análisis técnico de la falla, impacto, condiciones de explotación y una PoC guiada paso a paso.

⚠️ Disclaimer

Contenido con fines educativos. Respeta las normas de cada plataforma y licencias de terceros.

1 watching

2 forks

Releases

No releases published
Create a new release

Contributors 1

drovoh4k Llor

Languages

Python 76.4%
C 6.2%
Doc
Shell 3.5%

Suggested workf

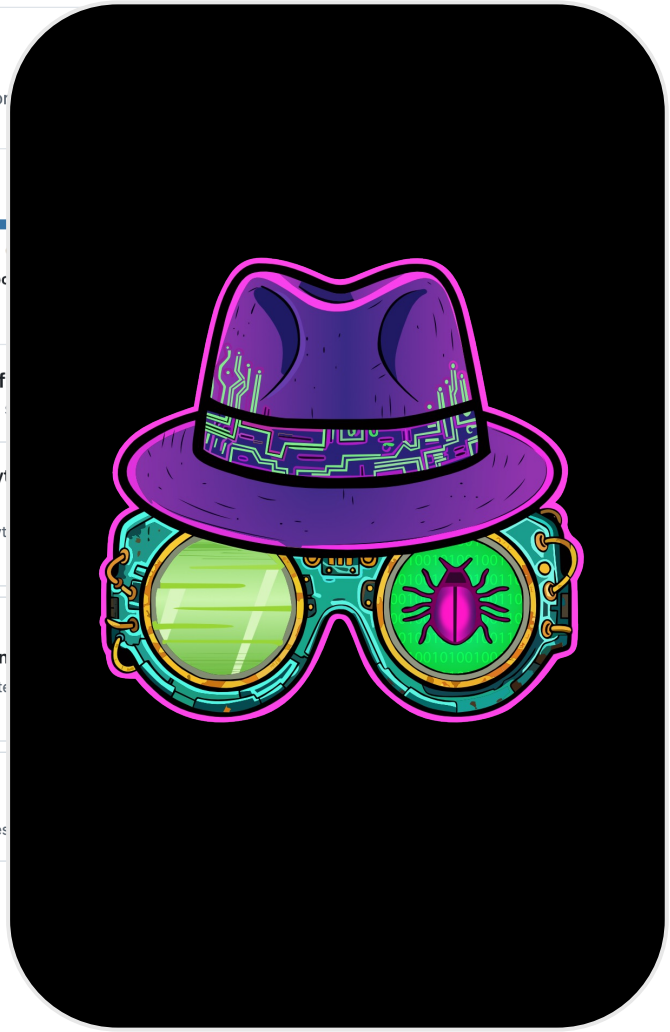
Based on your tech

Publish Py Package
Publish a Pyt on release.

Python application
Create and te application.

Django
Build and Tes

More workflows



SUSCRÍBETE

Drovohub